

Python: resumen de sintaxis

Documento de apoyo para el examen · IA Agéntica

Podrás usar este documento en el examen. Contiene la sintaxis y ejemplos, no las soluciones de los ejercicios. Léelo antes y, si crees que falta algo o que algo no está claro, propón cambios al profesor.

1. Lo básico

No hay main ni clase: el archivo se ejecuta de arriba abajo. La indentación (4 espacios) forma parte de la sintaxis.

```
# Esto es un comentario
print("Hola")
print("a", "b", "c")
print("a", "b", sep="-")
print("misma línea", end=" ")
nombre = input("Tu nombre: ")

# Hola
# a b c
# a-b
# no hace salto de línea
# siempre devuelve str
```

2. Variables y tipos

El tipo no se declara: lo decide el valor. La función `type()` dice cuál es.

Tipo	Ejemplo	Conversión
int	7, -3, 0	<code>int("7")</code>
float	3.5, 0.1	<code>float("3.5")</code>
str	"testua" / "texto"	<code>str(42)</code>
bool	True, False	<code>bool(0) → False</code>

Operadores

Operador	Significado	Ejemplo
+ - *	sumar, restar, multiplicar	<code>2 * 3 → 6</code>
/	división (siempre float)	<code>7 / 2 → 3.5</code>
//	división entera	<code>7 // 2 → 3</code>
%	resto	<code>7 % 2 → 1</code>
**	potencia	<code>2 ** 3 → 8</code>
== !=	igual, distinto	<code>3 == 3 → True</code>
< <= > >=	comparaciones	<code>5 > 2 → True</code>
and or not	lógicos	<code>True and False → False</code>
+= -=	sumar / restar y asignar	<code>x += 1</code>

f-strings

```
precio = 3.456
print(f"Precio: {precio} €")          # Precio: 3.456 €
print(f"Dos decimales: {precio:.2f}") # Dos decimales: 3.46
print(f"[{7:5}]")                    # [    7] ancho de 5
print(f"El año que viene: {20 + 1}") # entre llaves, cualquier expresión
```

3. Condicionales

```
if temperatura > 30:
    print("Calor")
elif temperatura > 15:
    print("Templado")
else:
    print("Frío")

if 18 <= edad < 65:          # las comparaciones se pueden encadenar
    print("Edad laboral")

estado = "abierto" if hora < 20 else "cerrado" # operador ternario

if "a" in "casa":           # in: ¿está dentro?
    print("Está")
```

Las cadenas se comparan con ==. Para no distinguir mayúsculas: texto.lower() == "hola".

4. Bucles

range	Qué genera
range(5)	0, 1, 2, 3, 4
range(2, 6)	2, 3, 4, 5
range(0, 10, 2)	0, 2, 4, 6, 8
range(5, 0, -1)	5, 4, 3, 2, 1

Ojo: el número final no se incluye. range(1, 11) va del 1 al 10.

```

for ciudad in ["Bilbao", "Vitoria", "San Sebastián"]:
    print(ciudad)

for i in range(3):
    print(i)                # 0, 1, 2

contador = 0
while contador < 3:
    contador += 1

for n in range(10):
    if n == 2:
        continue           # salta a la siguiente vuelta
    if n == 5:
        break               # sale del bucle
    print(n)                # 0, 1, 3, 4

for i, ciudad in enumerate(["Bilbao", "Vitoria"]):
    print(i, ciudad)        # 0 Bilbao / 1 Vitoria

for nombre, nota in zip(["Ane", "Iker"], [8, 6]):
    print(nombre, nota)     # dos listas a la vez

```

5. Funciones

```

def area(base, altura):
    return base * altura

print(area(4, 3))           # 12

def saludo(nombre="amigo"): # valor por defecto
    return f"Hola, {nombre}"

print(saludo())             # Hola, amigo
print(saludo("Ane"))        # Hola, Ane

def dividir(a, b):
    return a // b, a % b     # devuelve dos valores (tupla)

cociente, resto = dividir(17, 5) # desempaquetar: 3 y 2

```

Sin return, la función devuelve None. print no es lo mismo que return: print escribe en pantalla, return devuelve el valor.

6. Cadenas (str)

Método	Ejemplo	Resultado
len(s)	len("etxe")	4
s.lower() / s.upper()	"KaiXo".lower()	"kaixo"
s.strip()	" hola ".strip()	"hola"
s.split()	"a b c".split()	["a", "b", "c"]
s.split(",")	"1,2,3".split(",")	["1", "2", "3"]
sep.join(lista)	"-".join(["a", "b"])	"a-b"
s.replace(a, b)	"kaxa".replace("x", "s")	"kasa"

Método	Ejemplo	Resultado
s.capitalize()	"bilbo".capitalize()	"Bilbo"
s[0] / s[-1]	"python"[0]	"p"
s[1:4]	"python"[1:4]	"yth"

7. Listas y tuplas

```

fruta = ["manzana", "pera"]
fruta.append("uva")           # añadir al final
fruta.insert(0, "kiwi")       # añadir en una posición
fruta.remove("pera")          # borrar por valor
ultima = fruta.pop()          # sacar la última
print(fruta[0], len(fruta))    # índice y longitud
print("kiwi" in fruta)        # True

numeros = [5, 2, 8]
print(sorted(numeros))         # [2, 5, 8] lista nueva
numeros.sort()                 # ordena la original
print(*numeros)                # 2 5 8 (desempaquetada)

coordenadas = (43.2, -2.9)     # tupla: no se puede modificar

```

Comprensiones de listas

```

dobles = [x * 2 for x in [1, 2, 3]]      # [2, 4, 6]
grandes = [x for x in [4, 9, 1, 7] if x > 5] # [9, 7]
largos = [len(p) for p in ["sol", "casa"]] # [3, 4]

```

Se lee de izquierda a derecha: qué guardo, de dónde lo saco, con qué condición.

8. Diccionarios (dict)

```

libro = {"titulo": "Obabakoak", "año": 1988}
print(libro["titulo"])          # si la clave no existe: KeyError
print(libro.get("paginas", 0))  # si la clave no existe: 0
libro["paginas"] = 400          # añadir o modificar
del libro["año"]                # borrar
print("paginas" in libro)       # True

for clave, valor in libro.items():
    print(clave, valor)
print(list(libro.keys()))       # claves
print(list(libro.values()))     # valores

stock = {}
stock["boligrafo"] = stock.get("boligrafo", 0) + 5

cuadrados = {x: x ** 2 for x in range(4)} # {0: 0, 1: 1, 2: 4, 3: 9}

```

9. Conjuntos (set)

No admite duplicados y no tiene orden fijo. set() para crear uno vacío; {} es un diccionario vacío.

Operador	Significado	{1,2,3} y {2,3,4}
<code>a & b</code>	intersección: en los dos	{2, 3}
<code>a b</code>	unión: al menos en uno	{1, 2, 3, 4}
<code>a - b</code>	diferencia: en a pero no en b	{1}
<code>a ^ b</code>	diferencia simétrica: solo en uno	{1, 4}
<code>s.add(x)</code>	añadir un elemento	
<code>x in s</code>	¿está?	

10. Funciones útiles

Función	Ejemplo	Resultado
<code>len()</code>	<code>len([4, 1, 7])</code>	3
<code>sum()</code>	<code>sum([4, 1, 7])</code>	12
<code>max()</code> / <code>min()</code>	<code>max([4, 1, 7])</code>	7
<code>sorted()</code>	<code>sorted([4, 1, 7])</code>	[1, 4, 7]
<code>sorted(..., reverse=True)</code>	<code>sorted([4, 1, 7], reverse=True)</code>	[7, 4, 1]
<code>all()</code>	<code>all(x > 0 for x in [4, 1, 7])</code>	True
<code>any()</code>	<code>any(x > 5 for x in [4, 1, 7])</code>	True
<code>round()</code>	<code>round(3.14159, 2)</code>	3.14
<code>abs()</code>	<code>abs(-5)</code>	5

`sum()` puede contar los True: `sum(x > 3 for x in [4, 1, 7]) → 2`.

11. Módulos

```
import math           # math.sqrt(16)
import math as m      # m.sqrt(16)
from math import pi, sqrt # sqrt(16), directamente
import mi_modulo      # mi_modulo.py en la misma carpeta
```

Función	Qué hace
<code>math.sqrt(x)</code>	raíz cuadrada
<code>math.pi</code>	el número π
<code>math.floor(x)</code> / <code>math.ceil(x)</code>	redondear hacia abajo / arriba
<code>random.randint(a, b)</code>	un entero entre a y b, ambos incluidos
<code>random.uniform(a, b)</code>	un float entre a y b
<code>random.choice(lista)</code>	un elemento de la lista al azar
<code>string.punctuation</code>	todos los signos de puntuación

12. Manejo de errores

```
try:
    numero = int(input("Número: "))
    print(100 / numero)
except ValueError:
    print("No es un número")
except ZeroDivisionError:
    print("No se puede dividir entre cero")
finally:
    print("Se ejecuta siempre")

def comprobar_edad(edad):
    if edad < 0:
        raise ValueError("La edad no puede ser negativa")
    return edad
```

Error	Cuándo aparece
ValueError	conversión imposible: int("hola")
ZeroDivisionError	al dividir entre cero
KeyError	clave que no existe en el diccionario
IndexError	índice fuera de la lista
TypeError	tipos mezclados: "a" + 1
NameError	variable no definida
IndentationError	indentación incorrecta

`except Exception:` captura cualquier error. Es cómodo, pero también esconde errores que preferirías ver.